Kubernetes ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Kubernetes?
Kubernetes is a container orchestration platform used to deploy, manage, scale, and monitor containerized applications.
It automatically handles tasks like scheduling containers, restarting failed containers, scaling applications, and service discovery.
Simple ga gurthupettuko:
Docker = Container create/run
Kubernetes = Containers ni manage/orchestrate
Eng: Kubernetes is a container orchestration platform used to automate deployment, scaling, networking, and management of containerized applications.

---

2. Why do we need Kubernetes?
When we have many containers, manually managing them becomes difficult.
Kubernetes automates container deployment, scaling, service discovery, load balancing, self-healing, and rolling updates.
Simple ga gurthupettuko:
Many Containers → Kubernetes → Automatic Management
Eng: Kubernetes is used to manage large numbers of containers efficiently through automation and orchestration.

---

3. What is a Kubernetes Cluster?
A Kubernetes cluster is a group of machines that work together to run containerized applications.
It mainly contains a **control plane** and **worker nodes**.
Simple ga gurthupettuko:
Cluster = Control Plane + Worker Nodes
Eng: A Kubernetes cluster consists of control-plane components and worker nodes that together manage and run workloads.

---

4. What is a Kubernetes Node?
A node is a machine that runs application workloads in a Kubernetes cluster.
A node can be a physical machine or virtual machine.
The node contains components such as kubelet and a container runtime.
Simple ga gurthupettuko:
Node = Application containers run ayye machine.
Eng: A Kubernetes node is a worker machine where application workloads are scheduled and executed.

---

5. What is a Pod?
A Pod is the smallest deployable unit in Kubernetes.
It contains one or more closely related containers that share networking and storage resources.
Most commonly, one application container runs inside one Pod.
Simple ga gurthupettuko:
Pod = Kubernetes lo smallest deployable unit.
Eng: A Pod is the smallest deployable unit in Kubernetes and contains one or more containers that share network and storage resources.

---

6. What is the difference between Pod and Container?
A container is the actual application runtime.
A Pod is the Kubernetes abstraction that manages one or more containers.
Kubernetes schedules Pods rather than directly scheduling individual containers.
Simple ga gurthupettuko:
Container = Application runtime
Pod = Container(s) ni Kubernetes manage chese unit
Eng: A container runs the application, while a Pod provides the Kubernetes execution and networking context for one or more containers.

---

7. What is a Deployment?
A Deployment is a Kubernetes resource used to manage stateless applications.
It defines the desired number of Pod replicas and manages updates and rollbacks.
For example, if we specify three replicas, Kubernetes tries to keep three Pods running.
Simple ga gurthupettuko:
Deployment = Pods ni manage + scale + update.
Eng: A Deployment manages the desired state, replicas, rolling updates, and rollbacks of stateless application Pods.

---

8. What is a ReplicaSet?
A ReplicaSet ensures that a specified number of Pod replicas are running.
If a Pod fails, the ReplicaSet creates another Pod to maintain the desired count.
Deployments generally manage ReplicaSets rather than creating them directly for normal application deployment.
Simple ga gurthupettuko:
ReplicaSet = Desired number of Pods maintain chestundi.
Eng: A ReplicaSet maintains the desired number of matching Pods and replaces Pods when they fail.

9. What is a Kubernetes Service?
A Service provides a stable network endpoint for accessing a set of Pods.
Pods are temporary and their IP addresses can change, so applications should not depend directly on Pod IPs.
The Service selects Pods using labels and routes traffic to matching Pods.
Simple ga gurthupettuko:
Service = Stable endpoint for Pods.
Eng: A Kubernetes Service provides stable networking and load distribution to a set of Pods.

---

10. What are the main types of Kubernetes Services?
Common Service types are:

- ClusterIP
- NodePort
- LoadBalancer
- ExternalName

ClusterIP is mainly for internal communication.
NodePort exposes the service through a port on each node.
LoadBalancer can provision an external load balancer when supported by the environment.
Simple ga gurthupettuko:
ClusterIP = Internal
NodePort = Node port
LoadBalancer = External access
Eng: Kubernetes provides different Service types depending on whether the application needs internal or external connectivity.

---

11. What is ClusterIP?
ClusterIP is the default Kubernetes Service type.
It provides an internal virtual IP that allows applications inside the cluster to communicate with the Service.
It is not normally directly accessible from the public internet.
Simple ga gurthupettuko:
ClusterIP = Internal cluster communication.
Eng: ClusterIP exposes a Service internally within the Kubernetes cluster.

---

12. What is NodePort?
NodePort exposes a Service on a port across the cluster's nodes.
Traffic sent to that node port can be forwarded to the Service's backend Pods.
It is simple but generally not the preferred production exposure method when a cloud load balancer or Ingress is available.
Simple ga gurthupettuko:
NodePort = Node IP + Port → Service → Pods
Eng: NodePort exposes a Service through a port on each node and forwards traffic to the Service's Pods.

---

13. What is LoadBalancer Service?
LoadBalancer is used to expose a Kubernetes Service externally through a cloud or infrastructure load balancer when supported.
In cloud environments, Kubernetes can integrate with the cloud provider to provision or configure the external load-balancing infrastructure.
Simple ga gurthupettuko:
Internet → Cloud Load Balancer → Kubernetes Service → Pods
Eng: A LoadBalancer Service exposes an application externally through an infrastructure load balancer provided by the environment.

---

14. What is Ingress?
Ingress is a Kubernetes API resource used to define HTTP/HTTPS routing rules for external traffic.
It can route requests based on hostname or URL path to different Services.
An Ingress requires an **Ingress Controller** to actually process the traffic.
Simple ga gurthupettuko:
Ingress = HTTP/HTTPS routing rules.
Eng: Ingress defines HTTP/HTTPS routing rules that direct external traffic to Kubernetes Services through an Ingress Controller.

---

15. What is an Ingress Controller?
An Ingress Controller is the component that implements the actual Ingress traffic routing.
It watches Kubernetes Ingress resources and configures the underlying proxy or load-balancing system accordingly.
Examples include NGINX Ingress Controller and cloud-provider-specific controllers.
Simple ga gurthupettuko:
Ingress = Rules
Ingress Controller = Rules ni actually implement chestundi
Eng: An Ingress Controller processes Ingress resources and handles the actual HTTP/HTTPS traffic routing.

16. What is ConfigMap?
ConfigMap stores non-sensitive configuration data separately from the application image.
Examples include application settings, environment-specific values, and configuration parameters.
Pods can consume ConfigMap values as environment variables or mounted files.
Simple ga gurthupettuko:
ConfigMap = Non-sensitive configuration.
Eng: ConfigMap stores non-sensitive configuration data that can be provided to Kubernetes workloads without rebuilding the container image.

---

17. What is Kubernetes Secret?
A Secret is used to store sensitive information such as passwords, tokens, and credentials.
Pods can consume Secrets as environment variables or mounted files.
Secrets should still be handled carefully because Kubernetes Secret data is not automatically equivalent to a fully featured external secrets-management system.
Simple ga gurthupettuko:
Secret = Sensitive configuration.
Eng: Kubernetes Secrets provide a mechanism for storing and supplying sensitive configuration data to workloads.

---

18. Difference between ConfigMap and Secret?
ConfigMap is intended for non-sensitive configuration.
Secret is intended for sensitive information such as credentials and tokens.
Simple ga gurthupettuko:
ConfigMap = Normal config
Secret = Sensitive config
Eng: ConfigMaps store non-sensitive configuration, while Secrets are designed for sensitive configuration data.

---

19. What is a Namespace?
A Namespace provides a logical separation of resources inside a Kubernetes cluster.
It can be used to separate teams, environments, or applications and can also be used with access controls and resource policies.
For example:
dev
test
prod
Simple ga gurthupettuko:
Namespace = Cluster lo logical separation.
Eng: A Namespace provides logical isolation and organization of Kubernetes resources within a cluster.

---

20. What are Labels in Kubernetes?
Labels are key-value pairs attached to Kubernetes objects.
They are used to identify and organize resources and are especially important for selecting Pods.
For example:
app=payment
A Service can use this label to select the appropriate Pods.
Simple ga gurthupettuko:
Label = Resource identification.
Eng: Labels are key-value metadata used to organize resources and select matching objects.

---

21. What are Selectors in Kubernetes?
Selectors are used to identify Kubernetes objects based on labels.
For example, a Service can select all Pods with:
app=frontend
The Service then routes traffic to those matching Pods.
Simple ga gurthupettuko:
Selector = Matching Labels ni find chestundi.
Eng: Selectors identify Kubernetes resources by matching their labels.

---

22. What is a Kubernetes Volume?
A Volume provides storage that can be mounted into a Pod.
Different volume types have different lifecycles and persistence characteristics.
Volumes are useful when applications need storage beyond the lifetime of an individual container.
Simple ga gurthupettuko:
Volume = Pod/Container ki storage.
Eng: A Kubernetes Volume provides storage that can be mounted into containers running within a Pod.

---

23. What is PersistentVolume?
PersistentVolume, or PV, is a cluster-level storage resource provisioned for Kubernetes workloads.
It represents storage that can exist independently of an individual Pod.
For example, a PV may represent cloud block storage or network storage.
Simple ga gurthupettuko:
PV = Actual persistent storage resource.
Eng: A PersistentVolume represents persistent storage available to Kubernetes workloads independently of a specific Pod.

24. What is PersistentVolumeClaim?
PersistentVolumeClaim, or PVC, is a request for storage made by a workload.
The PVC can request characteristics such as storage capacity and access mode, and Kubernetes can bind it to a suitable PersistentVolume.
Simple ga gurthupettuko:
PVC = Application storage request.
PV = Storage resource.
Eng: A PersistentVolumeClaim is a workload's request for persistent storage that can be bound to a suitable PersistentVolume.

---

25. What are Liveness and Readiness Probes?
A **liveness probe** checks whether the container is still healthy enough to continue running.
If liveness checks repeatedly fail, Kubernetes may restart the container.
A **readiness probe** checks whether the application is ready to receive traffic. If it fails, the Pod can be removed from Service endpoints without necessarily restarting it.
Simple ga gurthupettuko:
Liveness = Should I restart?
Readiness = Should I send traffic?
Eng: Liveness determines whether a container should be restarted, while readiness determines whether the application should receive traffic.

---

26. What is a Startup Probe?
A startup probe is used for applications that require significant time to initialize.
Kubernetes can wait for the startup probe to succeed before applying normal liveness and readiness checks.
This prevents slow-starting applications from being restarted prematurely.
Simple ga gurthupettuko:
Startup Probe = Application startup complete ayinda?
Eng: A startup probe allows Kubernetes to determine whether a slow-starting application has completed initialization before enforcing normal health checks.

---

27. What is Kubernetes Self-Healing?
Kubernetes continuously compares the actual state with the desired state.
If a Pod fails, Kubernetes can create a replacement Pod through controllers such as Deployments and ReplicaSets.
This automatic recovery behavior is commonly called self-healing.
Simple ga gurthupettuko:
Pod fail → Kubernetes replacement create chestundi.
Eng: Kubernetes provides self-healing by detecting failed workloads and recreating them to maintain the desired state.

---

28. What is a Rolling Update?
A rolling update gradually replaces old Pods with new Pods.
This allows an application to be updated without stopping all replicas simultaneously.
Deployments use rolling updates as a common strategy for minimizing application downtime.
Simple ga gurthupettuko:
Old Pods gradually replace → New Pods.
Eng: A rolling update gradually replaces old application Pods with new ones to minimize disruption during deployment.

---

29. How do you troubleshoot a Pod that is not running?
First, I check the Pod status:
kubectl get pods
Then I inspect the Pod:
kubectl describe pod <pod-name>
I also check:
kubectl logs <pod-name>
Depending on the error, I investigate image pulling, configuration, Secrets, resource limits, probes, scheduling, or application startup issues.
Simple ga gurthupettuko:
get pods → describe → logs → Root Cause
Eng: I troubleshoot a failed Pod by checking its status, events, logs, image, configuration, resources, probes, and scheduling information.

---

30. What is CrashLoopBackOff?
CrashLoopBackOff means a container is repeatedly starting and then crashing, so Kubernetes is increasing the delay between restart attempts.
Common causes include application startup failures, incorrect configuration, missing environment variables, dependency failures, or incorrect commands.
I first check the Pod logs and describe output to identify the root cause.
Simple ga gurthupettuko:
Start → Crash → Restart → Crash → Backoff
Eng: CrashLoopBackOff indicates that a container repeatedly crashes and Kubernetes is applying increasing delays between restart attempts.

31. What is ImagePullBackOff?
ImagePullBackOff means Kubernetes could not pull the required container image and is backing off before trying again.
Common causes include an incorrect image name/tag, private registry authentication issues, or registry/network problems.
I check Pod events using kubectl describe pod and verify the image and registry credentials.
Simple ga gurthupettuko:
Image Pull Fail → Retry Delay
Eng: ImagePullBackOff indicates that Kubernetes is unable to pull the required container image and is delaying subsequent attempts.

---

32. How do you troubleshoot a Service that is not reaching Pods?
First, I check the Service:
kubectl get svc
Then I verify the Service selector and compare it with the Pod labels.
I also check the endpoints:
kubectl get endpoints
If endpoints are missing, I investigate label selectors, Pod readiness, and Service configuration.
Simple ga gurthupettuko:
Service → Selector → Pod Labels → Endpoints
Eng: I troubleshoot Service connectivity by checking selectors, Pod labels, readiness, endpoints, ports, and network policies.

---

33. How do you troubleshoot a Pod that cannot connect to another Pod?
First, I verify that both Pods are running and ready.
Then I check the Service name, Service port, target port, DNS resolution, network policies, and application listening port.
I also test connectivity from inside the Pod when appropriate.
Simple ga gurthupettuko:
Pod → Service/DNS → Port → Network Policy → Application
Eng: I troubleshoot Pod-to-Pod connectivity by checking DNS, Services, ports, readiness, network policies, and application listeners.

---

34. What is kubectl**?**
kubectl is the command-line tool used to communicate with the Kubernetes API server.
It is used to create, inspect, update, troubleshoot, and delete Kubernetes resources.
Examples:
kubectl get pods
kubectl describe pod <name>
kubectl logs <name>
Simple ga gurthupettuko:
kubectl = Kubernetes ni CLI nunchi manage cheyyadaniki tool.
Eng: kubectl is the Kubernetes command-line client used to interact with the Kubernetes API.

---

35. Explain a real-time Kubernetes deployment flow.
Developer code ni Git lo push chestadu.
CI pipeline application ni build chesi Docker image create chestundi and registry ki push chestundi.
Deployment process Kubernetes Deployment manifest ni update chestundi. Kubernetes new image tho Pods create chesi, health checks verify chesi, rolling update complete chestundi.
Simple ga gurthupettuko:
Git → CI/CD → Docker Build → Registry → Kubernetes Deployment → Pods → Service → Ingress → Users
Eng: In a typical DevOps flow, the CI/CD pipeline builds and pushes a container image, Kubernetes deploys the new version through a Deployment, verifies Pod health, and exposes the application through Services and Ingress.